

Sovereign Resource Manager Architecture Specification

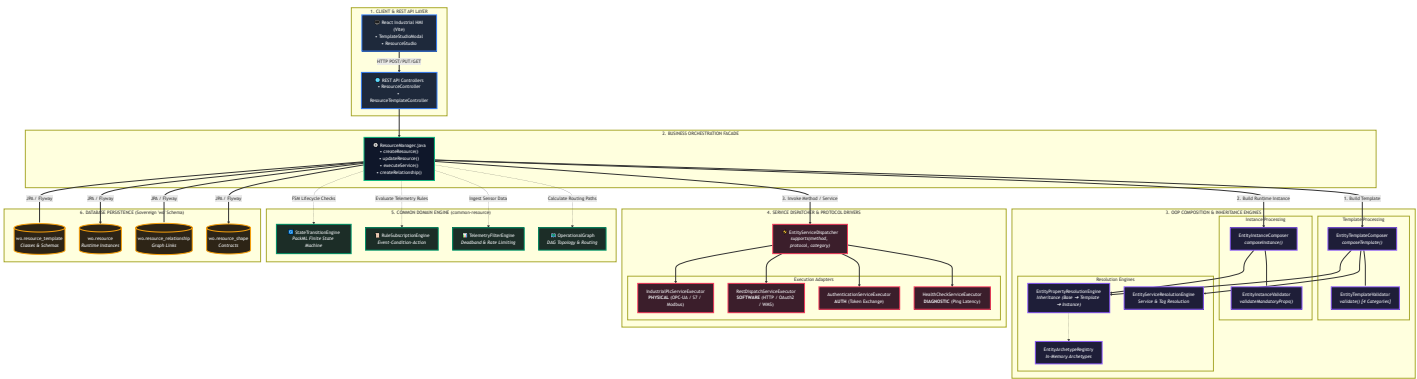
1. Executive Summary & Purpose

The **Resource Manager** is the foundational digital twin and asset orchestration engine of **Warehouse Orchestrator (WO)**. It decouples core industrial resource modeling, inheritance hierarchies, telemetry processing, finite state machines (FSM), and protocol dispatching from satellite execution software (WES, WMS, WCS, Fleet Management).

By isolating resource definitions and topology into a sovereign PostgreSQL schema (**wo**), the platform ensures:

- 1. **Satellite Decoupling:** WES, WMS, and WCS can be replaced, multiplied, or upgraded without altering physical asset definitions.
- 2. **True Industrial OOP Hierarchy:** Universal 3-tier property and service inheritance (*System Base -> Template Definition -> Instance Overrides*).
- 3. **Multi-Protocol OT Dispatching:** Transparent execution across physical equipment (**PHYSICAL**), external software connectors (**SOFTWARE**), digital twins (**VIRTUAL**), and logical routing nodes (**LOGICAL**).
- 4. **IEC 62443 Security & Safety Tiers:** Least-privilege action gating and dual-approval requirements for safety-critical hardware commands.

2. End-to-End Architectural Block Diagram



3. Detailed Component & Class Catalog

Layer 1: Inbound Web & Client Layer

(client/warehouse-ui)

File / Component	Purpose & Functionality
TemplateStudioModal.tsx	Main modal coordinator (< 600 lines) for authoring resource templates. Coordinates sub-tabs and triggers REST mutations.
TemplateIdentityTab.tsx	Manages code, name, universal category selection (PHYSICAL, SOFTWARE, VIRTUAL, LOGICAL), and OT Gateway communication method.
TemplateBasePropertiesTab.tsx	Displays universal platform properties (operational_state, health_status, fault_code, last_heartbeat) and default values.
TemplateCustomPropertiesTab.tsx	User-defined custom hardware tags across 12 industrial data types (STRING, DOUBLE, LOCATION, MAP, etc.).
TemplateBaseMethodsTab.tsx	Core safety and lifecycle operations (PING_HEALTH, RESET_FAULT, EMERGENCY_STOP).
TemplateCustomMethodsTab.tsx	Custom callable operations, IEC 62443 safety tiers, and supported direct command tags (START, STOP, JOG_FWD).
resourceTemplateService.ts	Frontend Axios/fetch client mapping CRUD requests to /api/v1/resource-templates.

Layer 2: Business Orchestration Facade

(services/wes-service/.../business/resource)

Class	Key Methods	Purpose
ResourceManager	- createResource() - updateResource() - deleteResource() - getResource() - getAllResources() - executeService() - createRelationship() - getDownstreamResources()	Central orchestration facade. Validates incoming requests, manages template inheritance, persists to PostgreSQL schema wo, and delegates execution calls to EntityServiceDispatcher.

Layer 3: Composition, Inheritance & Validation Engines (services/wes-service/.../composer)

Class	Key Methods	Purpose
EntityTemplateComposer	- composeTemplate() - validateTemplate()	Compiles user-defined property schemas and methods into a finalized ComposedEntityTemplate .
EntityInstanceComposer	- composeInstance() - resolveEffectiveState()	Assembles an active runtime ComposedEntityInstance by overlaying template defaults with instance runtime values.
EntityPropertyResolutionEngine	- resolveEffectiveProperties() - mergeDefaultProperties()	Executes 3-tier property inheritance: (1) <i>System Universal Base</i> -> (2) <i>Template Defaults</i> -> (3) <i>Instance Custom Overrides</i> .
EntityServiceResolutionEngine	resolveAvailableServices()	Determines the effective callable operations and supported commands for an asset instance.
EntityArchetypeRegistry	- getArchetype(code) - getAllArchetypes() - getByCategory(category)	In-memory registry maintaining pre-built factory archetypes (e.g., Generic REST, OPC-UA Device).
EntityTemplateValidator	- validate() - validateProperties() - validateMethods()	Enforces validation rules: ensures category is in [PHYSICAL, SOFTWARE, VIRTUAL, LOGICAL] , checks property key regex, and verifies data types.
EntityInstanceValidator	validateInstance()	Verifies that all mandatory fields (required: true) are satisfied before creating a runtime resource.
EntityComposerMapper	- toDto() - toEntity()	Maps between JPA database entities and

Class	Key Methods	Purpose
REST API DTOs.		

Layer 4: Protocol Dispatcher & Execution Drivers

(services/wes-service/.../service)

Class	Key Methods	Purpose
EntityServiceDispatcher	dispatch(instance, method, params)	Matches the target method, communication protocol, and asset category to an active driver via supports().
IndustrialPlcServiceExecutor	- supports(...) - execute(...)	Physical equipment driver (PHYSICAL). Communicates with OPC-UA, Siemens S7, and Modbus TCP for operations (READ_TAG, WRITE_TAG, TRIGGER_SCENARIO).
RestDispatchServiceExecutor	- supports(...) - execute(...)	Software connector driver (SOFTWARE). Executes dynamic HTTP GET/POST/PUT calls to external WMS/MES/ERP endpoints.
AuthenticationServiceExecutor	- supports(...) - execute(...)	Software authentication driver (AUTHENTICATE). Handles OAuth2 token retrieval and API key authentication.
HealthCheckServiceExecutor	- supports(...) - execute(...)	Universal diagnostic driver (PING_HEALTH). Validates network reachability, heartbeat timestamps, and round-trip latency.

Layer 5: Common Industrial Domain Engines

(common/common-resource)

Class	Key Methods	Purpose
StateTransitionEngine	transition(currentState, trigger, profile)	Finite State Machine (PackML / Semi E10 compliant) governing

Class	Key Methods	Purpose
	• <code>canTransition(from, to)</code>	asset lifecycle (<code>IDLE</code> , <code>STARTING</code> , <code>RUNNING</code> , <code>HOLDING</code> , <code>STOPPED</code>).
<code>RuleSubscriptionEngine</code>	• <code>evaluateRules(resource, telemetry)</code> • <code>registerSubscription(rule)</code>	Event-Condition-Action (ECA) rule evaluator (e.g. if AMR battery < 15%, auto-trigger <code>send_to_charger</code>).
<code>TelemetryFilterEngine</code>	• <code>filter(point, policy)</code>	High-throughput deadband and rate-limiting filter for sensor streams (prevents database flooding on micro-jitter).
<code>OperationalGraph</code>	• <code>addNode()</code> • <code>addEdge()</code> • <code>findDownstreamNodes()</code>	Directed Acyclic Graph (DAG) modeling plant topology, physical conveyors, and logical transfer queues.

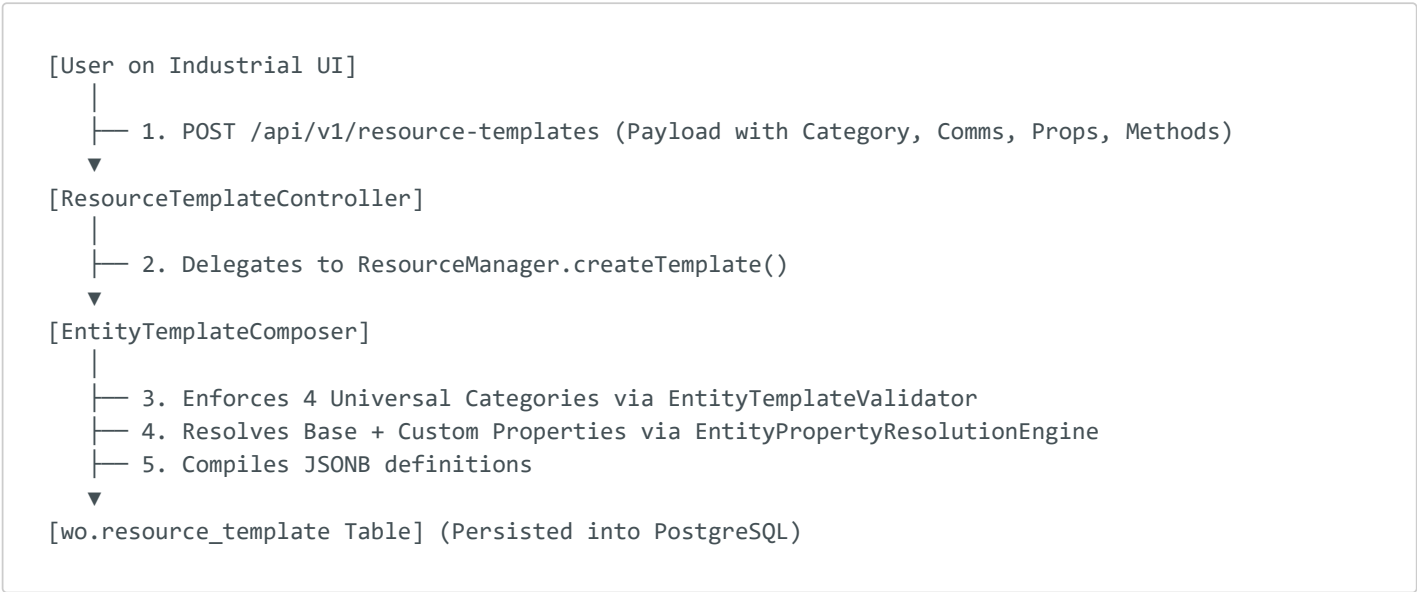
Layer 6: Persistence Layer (PostgreSQL `wo` Schema)

Table	Primary Columns	Purpose
<code>wo.resource_template</code>	<code>template_code</code> , <code>template_name</code> , <code>category</code> , <code>resource_type</code> , <code>communication_method</code> , <code>property_schema</code> (JSONB), <code>default_properties</code> (JSONB), <code>methods_schema</code> (JSONB), <code>supported_commands</code> (JSONB)	Master catalog defining reusable asset blueprints and industrial schemas.
<code>wo.resource</code>	<code>resource_id</code> , <code>name</code> , <code>category</code> , <code>type</code> , <code>status</code> , <code>template_code</code> , <code>custom_properties</code> (JSONB), <code>template_properties</code> (JSONB), <code>methods_config</code> (JSONB)	Runtime instances representing active physical machines, software connectors, or digital twins.
<code>wo.resource_relationship</code>	<code>source_resource_id</code> , <code>target_resource_id</code> , <code>relation_category</code> , <code>relation_type</code> , <code>properties</code> (JSONB)	Directed operational graph edges (<code>MATERIAL_FLOW</code> , <code>INFORMATION_FLOW</code>).

Table	Primary Columns	Purpose
wo.resource_shape	shape_code , shape_name , properties (JSONB), methods (JSONB)	Standalone interface contracts for cross-template interoperability.

4. End-to-End Operational Lifecycle & Sequences

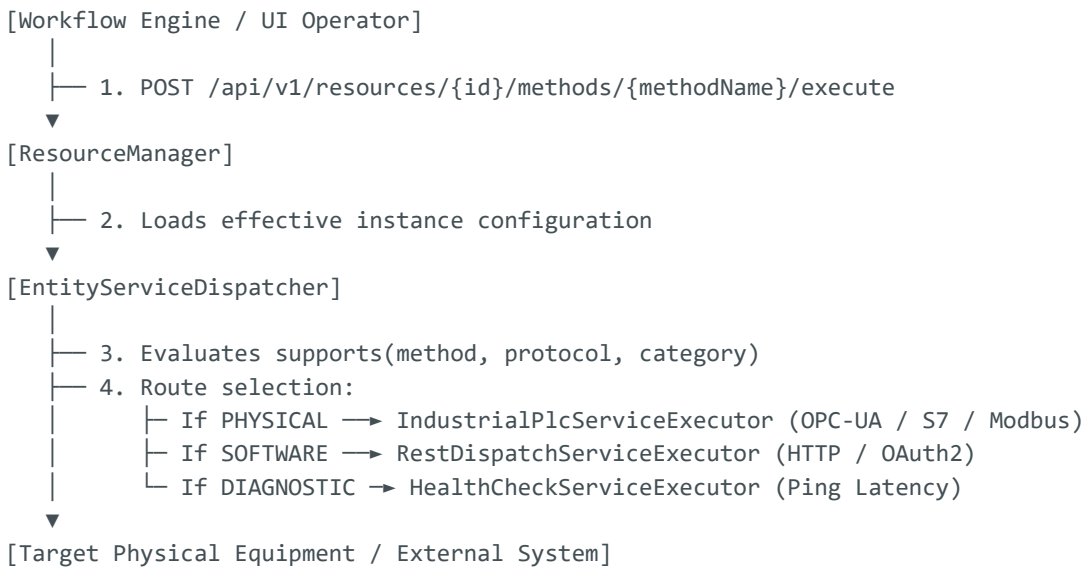
Flow 1: Template Creation (Design Time)



Flow 2: Resource Instantiation (Commissioning Time)



Flow 3: Method Execution / Service Dispatch (Runtime)



5. Refactoring Roadmap & Recommendations

To maintain clean modular separation between the platform core and satellite applications:

1. Move Facade & Composer to Platform Module:

- Extract `ResourceManager.java` and `composer/` out of `services/wes-service` into a dedicated platform module (`platform-resource-service` or directly inside `common/common-resource`).
- WES will consume this library as a clean client or injected service rather than hosting the core master data logic.

2. Clean Up Legacy Columns on `wo.resource`:

- Gradually deprecate top-level columns `host`, `port`, `protocol`, and `application` on `wo.resource`.
- All connection endpoints should reside natively in `custom_properties` or be resolved via the Industrial OT Gateway.

3. Domain Entity Consolidation:

- Eliminate redundant DTO/model mappings by adopting canonical domain models (`Resource`, `ResourceTemplate`, `PropertyDefinition`, `MethodDefinition`) from `common/common-resource` throughout all services.